

WES 237A: Introduction to Embedded System Design (Winter 2026)

Lab 4: Network Communication

Due: 2/16/2026 11:59pm

To report and reflect on your WES 237A labs, please complete this Post-Lab report by the deadline by submitting the following 2 parts:

- Upload your Lab 4 report, composed by a single PDF that includes your in-lab answers to the bolded questions in the Google Doc Lab and your Jupyter Notebook code. You could either scan your written copy or simply type your answer in this Google Doc. **However, please make sure your responses are readable.**
- Answer two short essay-like questions on your Lab experience.

All responses should be submitted to Canvas. Please also be sure to push your code to your git repo as well.

Locating IP Addresses of Devices in your Network

1. Connect the PYNQ board to the network switch over Ethernet.
2. Run '\$ ifconfig'. This is the *Interface Configuration* command and will tell you the different interfaces on your PYNQ board.
 - a. **How many IPv4 addresses are assigned to the board? What is the IPv4 address assigned to the 'eth0' or Ethernet interface? What is the netmask of this address?**
 - i. Note: 'eth0: 1' or 'usb0' is a virtual interface through the USB cable. This assigns your board an IP address over USB. This is a static IP address, so you can always reach your board from this IP address over USB.

```

Last login: Sat Feb 14 06:17:36 UTC 2026 on ttyPS0
xilinx@pynq:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.8.162 netmask 255.255.255.0 broadcast 192.168.8.255
    inet6 fe80::a48c:38ff:fe61:f62 prefixlen 64 scopeid 0x20<link>
    ether a6:8c:38:61:0f:62 txqueuelen 1000 (Ethernet)
    RX packets 40 bytes 3757 (3.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 107 bytes 12805 (12.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 40 base 0xb000

eth0:1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.99 netmask 255.255.255.0 broadcast 192.168.2.255
    ether a6:8c:38:61:0f:62 txqueuelen 1000 (Ethernet)
    device interrupt 40 base 0xb000

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 158 bytes 10314 (10.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 158 bytes 10314 (10.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

xilinx@pynq:~$

```

There is 2 network interfaces:

1. eth0 192.168.8.162
2. eth0:1 192.168.2.99

3. Connect your computer to the campus WiFi (UCSD-PROTECTED)
4. On your computer, open a command prompt and run `\$ ipconfig` on Windows and `\$ ifconfig` on MAC/Linux (it may take a second to connect, so wait a minute and then run the command)
 - a. **How many IPv4 addresses are assigned to this machine? What IPv4 address has the same netmask as the PYNQ board?**

En0: 100.81.38.107

5. Right now, your local machine and your PYNQ board form a network! However, we're more interested in networking two PYNQ boards together rather than your local machine and your PYNQ board. Luckily, every device hooked up to the switch is assigned an IP address on this network. That means we can communicate with any other board in the class. **Below, compile all the IP addresses of the PYNQ boards in your group.**

Pynq: 192.168.8.203 (ben)
Pynq: 192.168.8.162 (me)

6. To access your PYNQ board Jupyter notebooks, go to <PYNQ-IP>:9090

PYNQ-PYNQ Communication with Python

- Here, we're going to implement basic message-sending functionality in Python between two PYNQ boards.
- Download ['sockets_example.ipynb'](#)
- Go through and complete the code. Answer the following questions.
 - **What does `socket.SOCK_STREAM` mean (hint: search the documentation link in the notebook)?**

These constants represent the socket types, used for the second argument to `socket()`. More constants may be available depending on the system. (Only `SOCK_STREAM` and `SOCK_DGRAM` appear to be generally useful.) Specifically, TCP stream.

- **What is the order of operations for starting a client socket and sending a message?**

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((server, port))
sock.sendall(message)
```

1. Create socket
2. Setup ip and on port for connection
3. Send data to the socket

- **What is the order of operations for starting a server socket and receiving a message?**

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s: s.bind((ip, port))
s.listen(1) conn, addr = s.accept()
```

1. Create socket
2. Setup ip and on port for connection
3. Enable a server to accept connections.
4. Accept a connection

Wireshark

1. On your local machine (or lab machine), install [Wireshark](#)

If you are on Windows:

- a. Open the Firewall and Network Protection
- b. Click 'Allow an app through firewall.'
- c. Click 'Change Settings'
- d. Scroll down to 'Python'
- e. Select all 'Python' applications and all 'Public' boxes for each 'Python'

☒ Python	☒	☐	☒	No
☒ Python	☒	☐	☒	No

- f. Open the program 'IDLE (Python 3.7 64-bit)'

If you are on Linux / MacOS:

- g. Run the following command:
sudo ufw status
- h. If you get Active status, run this:
Sudo ufw allow 12345/tcp
- i. Run idle3 from the terminal (Linux) or search for IDLE in applications (Mac)

2. Click File->New File and paste the following code (**Check for tab v space errors when copying and pasting**)

```
import socket
import time
import signal
import sys

def run_program():
    sock_l = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock_l.bind(('0.0.0.0', 12345))
    sock_l.listen()
    print('Waiting for connection')
    conn, addr = sock_l.accept()
    print('Connected')
    with conn:
        data = conn.recv(1024)
        print(data.decode())

if __name__ == '__main__':
    original_sigint = signal.getsignal(signal.SIGINT)
    signal.signal(signal.SIGINT, exit)
    run_program()
```

3. Save the file, then select 'Run -> Run Module'. This is a slight variation of your server. It is waiting on port 12345 on the local lab machine.
4. From your PYNQ board, connect your client to
 - a. IP: local lab IP
 - b. Port: 12345

5. Send the message "Hello world\n"
6. You should see it displayed in the Python terminal
7. Now open Wireshark
8. Double click 'Wi-Fi' or 'Ethernet', depending on how you connected to the network.
You're now capturing a trace of the network traffic between your machine and the PYNQ board, via the router. Look at a few of the traces. Notice which are between your PYNQ board and the local machine (check the IP addresses) and which involve the router.
There will only be a difference if you also connect the PYNQ board directly to your local machine.
9. Where it says 'Apply a display filter' at the top, type 'tcp'
10. Repeat steps 9-11
11. Check the packet trace for any changes
 - a. **What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the PYNQ board and the local machine? What is it in the segment that identifies the segment as a SYN segment?**

USB 10/100 LAN: en6

No.	Time	Source	Destination	Protocol	Length	Info
5	3.448725	192.168.2.99	192.168.2.2	TCP	68	9090 → 54105 [PSH, ACK] Seq=1 Ack=1 Win=514 Len=2 TSval=3746633611 TSecr=...
6	3.448780	192.168.2.2	192.168.2.99	TCP	66	54105 → 9090 [ACK] Seq=1 Ack=3 Win=2048 Len=0 TSval=1001902120 TSecr=37...
7	3.448897	192.168.2.2	192.168.2.99	TCP	72	54105 → 9090 [PSH, ACK] Seq=1 Ack=3 Win=2048 Len=6 TSval=1001902120 TSe...
8	3.497127	192.168.2.99	192.168.2.2	TCP	66	9090 → 54105 [ACK] Seq=3 Ack=7 Win=514 Len=0 TSval=3746633660 TSecr=100...
10	5.150768	192.168.2.2	192.168.2.99	TCP	78	54182 → 12345 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSva...
11	5.151381	192.168.2.99	192.168.2.2	TCP	74	12345 → 54182 [SYN, ACK, ECE] Seq=0 Ack=1 Win=33304 Len=0 MSS=1460 SACK...
12	5.151435	192.168.2.2	192.168.2.99	TCP	66	54182 → 12345 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=2832001416 TSecr...
13	5.364124	192.168.2.99	192.168.2.2	TCP	696	9090 → 54105 [PSH, ACK] Seq=3 Ack=7 Win=514 Len=630 TSval=3746635526 TS...
14	5.364215	192.168.2.2	192.168.2.99	TCP	66	54105 → 9090 [ACK] Seq=7 Ack=633 Win=2039 Len=0 TSval=1001904036 TSecr=...
15	9.185848	192.168.2.2	192.168.2.99	TCP	85	54182 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=131776 Len=19 TSval=2832005451...
16	9.186455	192.168.2.99	192.168.2.2	TCP	66	12345 → 54182 [ACK] Seq=1 Ack=20 Win=33344 Len=0 TSval=3746639349 TSecr...
17	9.187040	192.168.2.99	192.168.2.2	TCP	66	12345 → 54182 [FIN, ACK] Seq=1 Ack=20 Win=33344 Len=0 TSval=3746639349...

Source: RealtekSemic_3b:0c:bb (00:00:00:00:00:00)
 Type: IPv4 (0x0800)
 [Stream index: 1]
 Internet Protocol Version 4, Src: 192.168.2.2, Dst: 192.168.2.99
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 Differentiated Services Field: 0x02 (DSCP: CS0, ECN: ECT(0))
 Total Length: 71
 Identification: 0x0000 (0)
 010. = Flags: 0x2, Don't fragment
 ...0 0000 0000 0000 = Fragment Offset: 0
 Time to Live: 64
 Protocol: TCP (6)

```

In [11]: import socket
import time
import signal
import sys

def run_program():
    sock_l = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock_l.bind(('0.0.0.0', 12345))
    sock_l.listen()
    print('Waiting for connection')
    conn, addr = sock_l.accept()
    print('Connected')
    with conn:
        data = conn.recv(1024)
        print(data.decode())

if __name__ == '__main__':
    original_sigint = signal.getsignal(signal.SIGINT)
    signal.signal(signal.SIGINT, exit)
    run_program()

Waiting for connection
Connected
helllllllloooooooooooooo
  
```

- Right-click this trace and select 'Follow->TCP Stream'
- Repeat a few times with different messages. Describe what's happening in the 5-10 steps of the TCP sequence for this communication. You can refresh your TCP flags [here](#).

- The client sends a SYN packet to the server to request a connection.
- The server responds with SYN ACK, saying it received the request and is ready.
- The client sends an ACK back. This completes the three step handshake and the connection is established.
- The client sends the message "Hello world" to the server.
- The server sends an ACK to confirm it received the message.
- If more messages are sent, the same send and acknowledge process repeats.
- When finished, the client sends FIN to close the connection.

8. The server responds with ACK, then sends its own FIN, and the client sends a final ACK. The connection is closed.

```
In [1]: import time
from pynq.overlays.base import Base0verlay
import socket

base = Base0verlay("base.bit")
btns = base.btns_gpio
leds = base.leds
```

Sockets

This notebook has both a client and a server functionality. One PYNQ board in the group will be the client and SENDS the message. Another PYNQ board will be the server and RECEIVES the message.

Server

Here, we'll build the server code to LISTEN for a message from a specific PYNQ board.

When we send/receive messages, we need to pieces of information which will tell us where to send the information. First, we need the IP address of our friend. Second, we need to chose a port to listen on. For an analogy, Alice expects her friend, Bob, to deliver a package to our back door. With this information, ALICE (server ip address) can wait at the BACK DOOR (port) for BOB (client ip address) to deliver the package.

Format of the information ipv4 address: ###.###.###.### (no need for leading zeros if the number is less than three digits) port: ##### (it could be 4 or 5 digits long, but must be >1024)

Use the socket documentation (Section 18.1.3) to find the appropriate functions

<https://python.readthedocs.io/en/latest/library/socket.html>

```
In [18]: # Create TCP socket
# TODO:
# 1: Bind the socket to the pynq board <CLIENT-IP> at port <LISTENING-PORT>
# 2: Accept connections
# 3: Receive bytes from the connection
# 4: Print the received message

import socket
import threading
import time

stop_event = threading.Event()
SERVER = "192.168.2.99"
LISTENING_PORT = 12345
```

```

def recv_loop(ip,port,stop_event):
    print("connecting...")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((ip, port))
        s.listen(1)
        conn, addr = s.accept()
        with conn:
            print('Connected by', addr)
            while not stop_event.is_set():
                data = conn.recv(1024)
                if not data:
                    break
                print(data.decode())
                time.sleep(.01)
            conn.close()
            s.close()
            print("connection closed.")

# def recv_loop(i,s,e):
#     print("on")
#     while not e.is_set():
#         pass
#     print("OFF")

def btn_listener(event, mask):
    print("button listener on.")
    while True:
        time.sleep(.1)
        if btns.read() & mask:
            print("stop event request")
            event.set()
            break;
    print("button listener done.")

btnListenerThread = threading.Thread(target=btn_listener, args=(stop_event,0))
btnListenerThread.start()

t = threading.Thread(target=recv_loop, args=(SERVER,LISTENING_PORT,stop_event))
t.start()

```

button listener on.
connecting...

Client

Now, we can implement the CLIENT code.

Back to the analogy, now we're interested in delivering a package to our friend's back door. This means BOB (client ip address) is delivering a package to ALICE (server ip address) at her BACK DOOR (port)

Remember to start the server before running the client code

```
In [19]: import socket
SERVER = "192.168.2.99" #"192.168.8.203"
PORT = 12345

# Create TCP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

import socket
import threading
import time

stop_tx_event = threading.Event()

def tx(HOST, PORT, evnt):
    # Echo client program
    print("transmitting...")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        s.sendall(b'Hello, world')
        data = s.recv(1024)
        print('Received', repr(data))

tt = threading.Thread(target=btn_listener, args=(stop_tx_event, 0b0010,), daemon=True)
tt.start()

t = threading.Thread(target=tx, args=('192.168.2.99', 12345, stop_tx_event, ), daemon=True)
t.start()
```

```
button listener on.
transmitting...
Connected by ('192.168.2.99', 57318)
Hello, world
```

On your server, you should see the message and then the server will shutdown! When we close a socket, both the client and the server are disconnected from the port.

Instead, change the function above to send 5 messages before closing.

The pseudocode looks like this

- connect the socket
- for i in range(5)
 - msg = input("Message to send: ")
 - send the message (msg)
- close the socket

```
In [16]: import socket
import time
import threading

import socket
import time
import threading
```

```
def tx_loop(server, port, stop_tx_event=None):
    message = b"Hello world!\n"
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((server, port))
        print(f"Connected to {server}:{port}")

        for i in range(5):
            sock.sendall(message)
            time.sleep(1)

    except Exception as e:
        print("TX error:", e)
        time.sleep(2)

    finally:
        sock.close()
        print("Socket closed")

ttt = threading.Thread(target=tx_loop, args=('192.168.2.99', 12345,), daemon=True)
ttt.start()
```

Connected to 192.168.2.99:12345
 Connected by ('192.168.2.99', 50730)
 Hello world!

Hello world!

Hello world!

Hello world!

Hello world!

Socket closed
 connection closed.

```
In [11]: import socket
import time
import signal
import sys

def run_program():
    sock_l = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock_l.bind(('0.0.0.0', 12345))
    sock_l.listen()
    print('Waiting for connection')
    conn, addr = sock_l.accept()
    print('Connected')
    with conn:
        data = conn.recv(1024)
        print(data.decode())

if __name__ == '__main__':
    original_sigint = signal.getsignal(signal.SIGINT)
```

```
signal.signal(signal.SIGINT, exit)
run_program()
```

Waiting for connection

Connected

helllllllooooooooooooo

In []: